

Decision-Rights Allocation: A Governance-Centric Perspective on LLM-Enabled Agentic Systems

Oliver Zhou

Lily Li

Abstract

LLM-enabled agentic systems are increasingly deployed inside task-and-decision processes that retrieve information, reason over context, call tools, coordinate with other agents, and execute actions on behalf of humans. Existing conceptual work has usefully classified these systems by external behavior, architectural components, cognitive design patterns, autonomy levels, and multi-agent governance forms. Those frameworks clarify what agentic systems contain and how they behave, but they often leave implicit a design choice that becomes central once stochastic language models are connected to organizational processes: which party has the effective right to make each consequential decision? This paper proposes decision-rights allocation as a compact internal-mechanism perspective for understanding and designing LLM-enabled agentic systems. We define an agentic system as an allocation of decision rights among three parties: human actors, deterministic components, and stochastic LLM components. The distinction matters because LLMs do not merely add another software module; they introduce open-ended semantic judgment into information selection, reasoning, action selection, validation, and memory update. The framework interprets common design patterns—including retrieval-augmented generation, episodic memory, human-in-the-loop validation, domain-expert knowledge codification, provenance tracing, reinforcement learning, routing, schema validation, and ontology-derived semantic context—as mechanisms that shift decision rights across these three parties. We further propose a simple optimization model in which the desirable allocation depends on task risk, adaptability needs, scalability pressure, auditability requirements, deterministic codifiability, human capacity, model reliability, verification availability, and the cost of errors at scale. The contribution is a unifying vocabulary for comparing agentic-system designs and a first-step analytical model for deciding when LLMs should decide, when deterministic infrastructure should constrain them, and when humans should retain or regain control.

Introduction

LLM-enabled agentic systems have moved quickly from demonstrations to applied workflow designs¹. Recent examples include autonomous anomaly management in complex systems (Barenji and Khoshgoftar 2025), cybersecurity operations (Lazer et al. 2026), business-process management (Dumas, Milani, and Chapela-Campa 2026), mobile-network RAN optimization (Pellejero et al. 2025), software testing (Naqvi, Baqar, and Mohammad 2026; Hariharan 2025), education (Yu et al. 2025), agricultural recommendation

¹The term *workflow* is useful in this paper only in this broad sense. It does not mean a fully predefined process script. Many LLM-enabled agents are attractive precisely because they can construct, revise, or adapt sequences of action rather than simply execute fixed rules. When this paper refers to a workflow, it therefore means an enacted task-and-decision process: a sequence of consequential states, information choices, reasoning steps, tool calls, approvals, memory updates, and external actions that may be partly predefined, partly learned, and partly generated at run time. This broader usage is consistent with recent agentic-AI literature that treats agentic systems as moving beyond static automation while still operating inside operational processes, roles, and environments (Sapkota, Roumeliotis, and Karkee 2025; Lamo Castrillo et al. 2025; Milosevic and Rabhi 2026; Dumas, Milani, and Chapela-Campa 2026).

([Yamaguchi et al. 2025](#)), engineering knowledge reuse ([Son et al. 2026](#)), workplace task automation ([Z. Z. Wang et al. 2025](#)), and credit decisioning ([Kubam 2025](#)). Across these settings, the relevant system is no longer only a language model that produces a text response. It is a socio-technical arrangement in which LLMs interpret requests, select evidence, plan steps, call tools, write code, update memory, route subtasks, ask for human input, and sometimes trigger actions in external systems.

The conceptual literature has developed in parallel, but its main categories answer questions that are different from the one faced by a designer who must decide where authority should sit inside a deployed process. Component frameworks ask what modules an LLM agent contains, such as perception, reasoning and planning, memory, and execution ([Lamo Castrillo et al. 2025](#); [L. Wang et al. 2023](#)). Cognitive design-pattern work asks which recurring functions, such as observe–decide–act, episodic memory, commitment, reconsideration, and knowledge compilation, appear across agentic systems ([Wray, Kirk, and Laird 2025](#)). Typological work asks how agentic a system is along dimensions such as knowledge scope, perception, reasoning, interactivity, operation, contextualization, self-improvement, and normative alignment ([Wissuchek and Zschech 2025](#)). Agent-community frameworks ask how multiple agents and humans can be formally coordinated through roles, obligations, permissions, prohibitions, and accountability chains ([Milosevic and Rabhi 2026](#)). These frameworks are valuable because they make agentic systems describable.

Description alone, however, does not settle a governance-relevant design problem: who has the right to decide at each consequential point in the process? When a system retrieves documents, is the LLM deciding what evidence matters, or is a deterministic retriever constraining the evidence space? When an agent proposes a credit decision, a test case, a clinical-trial match, or an anomaly response, is the final decision made by the LLM, a rule engine, a human reviewer, or some combination of the three? When an agent learns from feedback or writes to memory, who decides what becomes persistent knowledge? These questions are not merely implementation details. They determine reliability, accountability, scalability, transparency, cost, and organizational control.

This paper proposes decision-rights allocation as an internal-mechanism perspective for LLM-enabled agentic systems. The core claim is that many design differences can be interpreted as different allocations of decision rights among three parties: deterministic components, stochastic LLM components, and human actors. Deterministic components include rules, schemas, retrieval algorithms, databases, validators, routers, calculators, workflow engines, formal ontologies, and other bounded control surfaces. Stochastic LLM components include generative models used for interpretation, reasoning, planning, summarization, action proposal, and natural-language interaction. Human actors include users, operators, domain experts, reviewers, auditors, managers, and affected stakeholders.

The perspective is intentionally compact. It does not replace component-level taxonomies, autonomy scales, cognitive architectures, or system-theoretic governance frameworks. Instead, it asks a more specific question: for each consequential step in an agentic task-and-decision process, which party holds goal-framing, information-selection, reasoning, action-selection, validation, and memory-update rights? Under this lens, common design patterns become rights-shifting mechanisms. Retrieval-augmented generation can reduce the LLM’s right to invent evidence by giving retrieval infrastructure the right to select or constrain context. Human-in-the-loop validation gives humans veto or approval rights. Domain-expert knowledge codification shifts recurring expert judgment from humans into reusable protocols that guide or constrain LLM outputs.

Provenance tracing creates inspectability and contestability, thereby strengthening ex post human and deterministic governance. Episodic memory, reinforcement learning, routing, schema validation, and ontology-derived semantic context similarly shift rights over what the agent recalls, how it adapts, which components it uses, which actions are valid, and which concepts it is allowed to use.

The paper makes three contributions. First, it reframes LLM-enabled agentic systems as allocations of decision rights among deterministic, stochastic, and human parties. Second, it provides a matrix that interprets common agentic-AI design patterns according to how they shift decision rights. Third, it proposes a first-step analytical model for choosing the desirable allocation of rights as a function of task risk, adaptability, scalability, transparency, codifiability, human capacity, model reliability, and verification availability. The goal is not to provide a final quantitative theory, but to offer a shared vocabulary for comparing designs and for asking why a particular system grants too much or too little decision authority to the LLM.

Existing conceptual foundations for agentic systems

What existing frameworks explain

Higher-level frameworks are needed because the vocabulary of agentic systems is unstable. A wide range of terms is now used to describe AI systems that act on behalf of humans: AI agents ([Sapkota, Roumeliotis, and Karkee 2025](#)), LLM-based agents ([Lamo Castrillo et al. 2025](#); [Milosevic and Rabhi 2026](#)), prompt-driven generative AI ([Lazer et al. 2026](#)), agentic workflow systems ([Kamalov et al. 2025](#); [Fiorini et al. 2025](#)), agents augmented with codified expert knowledge ([Uulu et al. 2026](#)), agentic AI ([Sapkota, Roumeliotis, and Karkee 2025](#); [Barenji and Khoshgoftar 2025](#); [Dumas, Milani, and Chapela-Campa 2026](#); [Lazer et al. 2026](#)), agentic systems ([Hu, Lu, and Clune 2024](#)), agentic LLM systems ([Wray, Kirk, and Laird 2025](#)), and agentic communities ([Milosevic and Rabhi 2026](#)). This terminology reflects a real expansion in system capability, but it also creates comparison problems. A chatbot with tool access, a multi-agent orchestration platform, a workflow recommender with episodic memory, and a governed enterprise agent community may all be called agentic, even though their scope, control structure, and decision locus differ substantially.

One family of frameworks resolves this problem by describing agentic systems at the component level. Such work identifies modules for perception, memory, reasoning, planning, and execution, and it explains how these modules interact in LLM-agent architectures ([Lamo Castrillo et al. 2025](#); [L. Wang et al. 2023](#)). This component view is useful because it prevents the agent from being treated as a black box. Its limitation for the present argument is that a component label does not specify which party controls the component at run time. A memory module may be curated by humans, written by the LLM, constrained by deterministic provenance rules, or shared across agents through a cross-framework knowledge base ([Tang et al. 2025](#)). The governance implication differs across these cases even when the architecture diagram still contains the same module.

A second family of frameworks describes recurring functions or patterns. Cognitive design-pattern work connects LLM agents to traditions such as BDI, Soar, and ACT-R, showing how observe–decide–act, commitment, reconsideration, episodic memory, procedural memory, semantic memory, and knowledge compilation recur across intelligent systems ([Wray, Kirk, and Laird 2025](#); [Laird 2022](#)). Agentic workflow research similarly highlights reflection, planning, tool use, and multi-agent collaboration as reusable workflow

paradigms (Kamalov et al. 2025). These patterns explain what kind of control function the system performs. They do not by themselves determine whether that function should be exercised by the LLM, by deterministic infrastructure, by a human, or by a hybrid arrangement.

A third family of frameworks classifies agency, autonomy, and governance. Agenticness typologies compare systems by dimensions such as knowledge scope, perception, reasoning, interactivity, operation, contextualization, self-improvement, and normative alignment (Wissuchek and Zschech 2025). Autonomy frameworks distinguish capability from operational control and propose user-centered levels such as operator, collaborator, consultant, approver, and observer (Feng, McDonald, and Zhang 2025). Other work proposes code-inspectable autonomy attributes (Cihon et al. 2025), data-agent autonomy levels that track the progressive transfer of dominance and responsibility (Y. Zhu et al. 2025), and observation-based measures of autonomy for in-the-wild systems (Pittman 2024). Agent-community frameworks go further by making roles, protocols, obligations, permissions, prohibitions, and accountability chains explicit in multi-agent human-AI settings (Milosevic and Rabhi 2026). These frameworks show that autonomy is not simply a technical byproduct of better models; it is a design property that must be specified, constrained, and evaluated.

The remaining gap: authority over process functions

The frameworks above solve important descriptive problems, but they leave a mechanism-level question unresolved. They can tell us that a system has memory, uses tools, reflects on outputs, coordinates multiple agents, or operates with some degree of autonomy. They do not always tell us which party has effective authority over the process function in a particular deployment. This distinction matters because the same pattern can allocate authority differently. RAG can mean that a deterministic retriever constrains the evidence set, or it can mean that the LLM writes the query, chooses sources, and resolves conflicts. Human-in-the-loop can mean that a human receives enough provenance to challenge upstream reasoning, or it can mean that the human sees only a polished final answer and is effectively asked to rubber-stamp the LLM’s framing.

The design problem is therefore not only whether a system is agentic, but how its agentic functions are governed. A system-level label such as *autonomous*, *human-overseen*, *agentic community*, or *multi-agent workflow* is too coarse when the practical question is whether the LLM should select evidence, write memory, trigger tools, route subtasks, or decide when escalation is necessary. The same aggregate autonomy level can hide very different internal allocations: one system may let the LLM control evidence selection and action proposals while a human approves the final result; another may use deterministic retrieval, schema-constrained tool calls, and human approval before irreversible actions. Both may be called human-in-the-loop, but their reliability and accountability profiles differ.

Decision-rights allocation enters at this point in the argument. It does not replace the existing frameworks; it uses them as inputs. Component frameworks identify the process functions that can be governed. Pattern frameworks identify recurring mechanisms that can move authority across functions. Autonomy and agent-community frameworks explain why oversight, accountability, and role definition matter. Decision-rights allocation adds the missing question: at each consequential node, which party has the effective right to frame the goal, select information, reason, choose an action, validate the result, or update memory?

Why LLM-enabled agentic systems require a distinct design lens

The need for a distinct lens does not rest on the claim that only LLM-enabled systems use tools, retrieve data, write records, or operate across multiple steps. Deterministic software, rule engines, workflow platforms, and classical machine-learning pipelines can also perform these functions. The distinctive governance issue is narrower: LLM-enabled agentic systems insert open-ended semantic judgment into control functions that were previously specified by fixed code, structured models, or human operators. Once that semantic judgment is connected to tools, memory, and organizational processes, it becomes necessary to ask not merely what the system can do, but which party has authority over each decision.

The first relevant feature is not stochasticity by itself, but **stochastic semantic discretion**. Classical machine-learning systems may also be statistical, but their outputs are often restricted to predefined labels, scores, or decisions. LLMs generate language over an open semantic space and can use that language to interpret goals, infer missing context, select evidence, formulate intermediate conclusions, and propose actions. This makes the boundary between interpretation and decision less visible. A deterministic controller may make a wrong decision, but its policy can often be replayed as fixed code or a fixed rule table. An LLM-generated decision is produced through a high-dimensional learned distribution whose failure modes may change with prompt, context, sampling setting, model version, and surrounding tools. Hallucination is therefore not only an information-quality problem; when the output becomes an input to a process, unsupported or contradictory content can reallocate information-selection and reasoning rights to the model without an explicit governance decision (Kiprono 2025; Taparia et al. 2026; Gumaan 2025).

Deterministic decoding does not eliminate this issue. Greedy decoding can collapse a conditional output distribution into a reproducible trajectory, which is useful for debugging, regression testing, and replay. Yet it does not make the model’s semantic judgment equivalent to a rule engine, schema validator, or calculator. It may also hide uncertainty, alternative valid solutions, and harmful completions in non-argmax regions of the output distribution (Joshi et al. 2026). In decision-right terms, the LLM still holds semantic judgment rights even when the sampling procedure is made reproducible.

The second relevant feature is **language-to-action coupling**. Deterministic systems can call tools, but their tool calls are usually generated by predefined program logic or bounded policies. In LLM-enabled agents, natural-language interpretation can be transformed into API calls, code execution, record updates, messages to users, or process transitions. Cao et al. (2026) describes this as an architectural mismatch between stochastic, often unstructured LLM outputs and backend systems that require deterministic, schema-conformant inputs. Contract-driven development, policy projection, sandboxing, and constrained action spaces are therefore not merely engineering conveniences. They are mechanisms for moving action-selection and execution rights away from unconstrained generation and into deterministic control surfaces.

The third feature is **generated context as future state**. Databases and deterministic workflow systems also store and retrieve information, so memory is not unique to LLM agents. What changes in LLM-enabled systems is that the model may semantically compress prior interactions, decide what counts as a reusable trace, retrieve analogous precedents, or convert feedback into future prompts and policies. Episodic-memory systems, graph-memoized reasoning, cross-framework agent memory, standardized workflow reuse, and reflective memory consolidation all make prior traces part of later reasoning (Fiorini et al. 2025; Singh

2025; Tang et al. 2025; Gaoyang Zhang et al. 2026; J. Wang 2025). This creates a specific governance problem: a false summary, misclassified trace, or poorly validated workflow segment can become a premise for future action. Memory-update rights therefore require more explicit governance than one-shot generation rights.

The fourth feature is **de facto authority under nominal oversight**. In many deployed systems, humans may formally approve outputs, but the LLM has already selected the evidence, framed the options, summarized uncertainty, and proposed the action. If the human cannot inspect those upstream decisions, formal oversight may not equal effective control. System-level interpretability work makes the same point from another angle: agentic failures can propagate across perception, reasoning, planning, memory, tool execution, and inter-agent coordination, so local explanations of isolated model outputs are insufficient for accountability (J. Zhu et al. 2026). Decision-rights allocation addresses this problem by decomposing oversight into specific rights over framing, evidence, reasoning, action, validation, and memory.

These features explain why existing foundations, while necessary, are not enough. Component taxonomies identify modules, but they do not specify who controls the module at a particular decision node. Cognitive design patterns identify recurring functions, but the same pattern can be implemented with LLM-dominant planning, deterministic policy constraints, or human approval. Agentiveness typologies describe how much agency a system appears to exhibit, but not which party should hold each right in a given deployment. Autonomy scales describe independence, but they do not decompose independence into evidence selection, reasoning, action selection, validation, and memory update. Oversight labels identify broad human-control arrangements, but a system can appear human-in-the-loop while still granting the LLM de facto rights over framing, evidence selection, and interpretation.

The same point applies to human oversight. Risk-proportionate oversight frameworks distinguish Human-in-Command, Human-in-the-Loop, and Human-on-the-Loop arrangements according to model influence and decision consequence (Kandikatla and Radeljic 2025). Decision-rights allocation provides a mechanism-level interpretation of these categories. Human-in-Command is not merely more oversight than Human-on-the-Loop; it means humans retain goal-framing and ultimate validation rights. Human-in-the-Loop may mean humans hold validation rights but not information-selection rights. Human-on-the-Loop may mean deterministic monitoring holds routine validation rights while humans retain escalation rights. Thus, oversight labels become more precise when decomposed into rights.

Decision-rights allocation framework

Three parties and component boundaries

We define an LLM-enabled agentic system as a task-and-decision process in which effective rights are allocated among three parties. A right is *effective* when a party can materially shape the result of a decision node, even if another party formally owns the process or signs off on the final output.

The first party is the **human**. Human actors include end users, domain experts, workflow owners, compliance officers, reviewers, auditors, managers, and affected stakeholders. Humans can provide goals, tacit knowledge, preferences, ethical judgment, contextual interpretation, validation, escalation decisions, accountabil-

ity, and feedback. Human decision rights are valuable when tasks require responsibility, contextual judgment, legitimacy, and interpretation of ambiguous social or organizational constraints. They are costly when they create bottlenecks, slow response, introduce inconsistency, or make scaling difficult.

The second party is the **deterministic component**. Deterministic components include conventional software, rule engines, schemas, calculators, databases, retrievers, bounded routers, workflow engines, validators, knowledge graphs, ontologies, and policy constraints. Their rights are valuable when repeatability, auditability, cost control, formal compliance, and replayability matter. They are limited when the environment is too open-ended, when rules are incomplete, when knowledge changes quickly, or when tacit expertise cannot be fully specified.

The third party is the **stochastic LLM component**. The LLM can interpret natural language, generate plans, summarize evidence, draft outputs, reason over ambiguous context, translate between formats, and adapt to novel cases. LLM decision rights are valuable when tasks require flexibility, semantic understanding, rapid adaptation, and scalable handling of heterogeneous inputs. They are risky when outputs are unreliable, difficult to audit, sensitive to prompt injection, connected to irreversible external actions, or cheap to generate but costly to verify and remediate at scale.

Figure 1 summarizes this view. The framework separates the parties that may hold authority from the process functions over which authority is exercised. The central design question is not whether a system contains humans, software, and an LLM, but how much effective decision right each party holds at each consequential node.

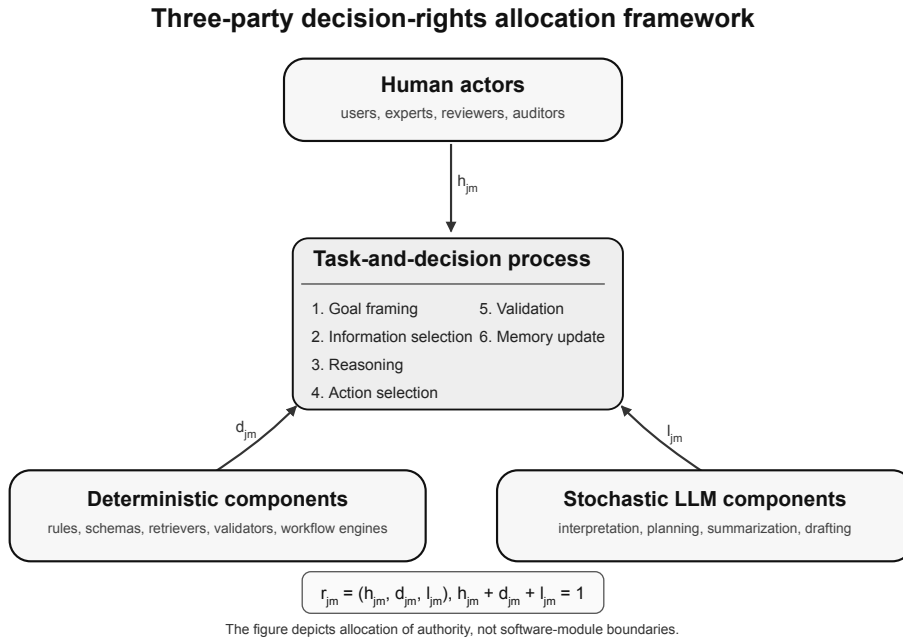


Figure 1: Three-party decision-rights allocation framework.

The boundary between deterministic and stochastic components deserves explicit treatment because many

governance errors arise from treating every system part around the LLM as if it were the LLM itself. Prompt templates, orchestration code, retrieval policies, schema constraints, output validators, approval gates, and workflow engines are not the LLM component even though they shape LLM behavior. They are deterministic or semi-deterministic control infrastructure around the LLM. Conversely, the LLM component should be understood as the model or model-service output at the system boundary. In practice, an external LLM API provider may use hidden deterministic tools, safety filters, retrieval systems, or post-processing steps. If these mechanisms are not observable or controllable by the system designer, the deployed system cannot allocate explicit deterministic rights to them. Their effect appears only as an empirical change in the reliability and behavior of the LLM service. If the provider exposes a tool, retrieval source, validator, or contract as a controllable interface, then that exposed interface can be modeled separately as deterministic infrastructure.

The three parties should not be understood as mutually exclusive software modules. A human may create a deterministic protocol that guides the LLM. An LLM may propose a rule that a human later approves. A deterministic router may choose which LLM to call. A human may validate an ontology-derived representation before it is used by the agent. A learned router may behave deterministically at runtime while still being estimated from data; in this framework it counts as deterministic infrastructure only to the extent that its action set, decision rules, confidence thresholds, and logs are bounded and auditable. The point of the framework is to ask which party has effective authority over each decision, not which party appears in the software diagram.

Decision rights within a task-and-decision process

A task-and-decision process can be decomposed into decision nodes. At each node, six primary types of rights may be allocated:

1. **Goal-framing rights:** who defines the objective, constraints, and acceptable trade-offs?
2. **Information-selection rights:** who decides what evidence, memory, context, or tool output is relevant?
3. **Reasoning rights:** who interprets the evidence and derives intermediate conclusions?
4. **Action-selection rights:** who chooses the next tool call, process transition, external action, or final recommendation?
5. **Validation rights:** who approves, rejects, revises, escalates, or audits the output?
6. **Memory-update rights:** who decides what should be stored, reused, revised, or forgotten?

The six categories are synthesized from the existing foundations rather than introduced as an arbitrary list. Component frameworks motivate information-selection, reasoning, action-selection, and memory-update rights because they identify perception, reasoning/planning, execution, and memory as core agent functions (Lamo Castrillo et al. 2025; L. Wang et al. 2023). Cognitive design-pattern work motivates goal-framing, action-selection, validation, and memory-update rights because observe–decide–act, commitment, reconsideration, episodic memory, and knowledge compilation each involve deciding what to attend to, what to commit to, when to revise, and what to preserve (Wray, Kirk, and Laird 2025; Laird 2022). Agenticness typologies motivate separating knowledge scope, reasoning, operation, contextualization, self-improvement, and normative alignment into more operational rights categories (Wissuchek and Zschech 2025). Human-oversight frameworks motivate validation and escalation rights because they distinguish human command,

intervention, and monitoring according to risk and consequence (Kandikatla and Radeljic 2025). Enterprise and agent-community frameworks motivate goal-framing and accountability-sensitive validation because they emphasize roles, protocols, obligations, and governance in multi-participant settings (Milosevic and Rabhi 2026).

These six categories are intended as primary rights rather than an exhaustive vocabulary of every operational subright. Terms such as representation rights, component-selection rights, execution rights, feedback rights, and context-construction rights can be mapped onto the six categories. For example, representation rights are a form of information-selection and reasoning rights; routing rights are a form of action-selection rights; schema-based execution rights are a constrained form of action-selection and validation rights; user-feedback incorporation is a form of validation and memory-update rights. This mapping keeps the framework compact while allowing applied analyses to use more granular labels.

Let a task-and-decision process contain decision nodes indexed by $j = 1, \dots, J$ and right types indexed by $m = 1, \dots, M$. For each pair (j, m) , define a decision-right vector

$$r_{jm} = (h_{jm}, d_{jm}, l_{jm}), \quad h_{jm} + d_{jm} + l_{jm} = 1, \quad h_{jm}, d_{jm}, l_{jm} \geq 0,$$

where h_{jm} is the share of effective decision right held by humans, d_{jm} is the share held by deterministic components, and l_{jm} is the share held by the LLM. This is not meant to imply that rights are always literally divisible. It is a measurement abstraction. A step in which the LLM drafts a recommendation but a human must approve it may have high LLM reasoning rights but high human validation rights. A step in which a schema blocks invalid tool calls gives deterministic components high action-selection and validation rights even if the LLM proposes the action.

The full object of design is the rights matrix $\{r_{jm}\}_{j,m}$. The aggregate system profile is a summary, not a substitute for the matrix:

$$R = \sum_{j=1}^J \sum_{m=1}^M \omega_j \lambda_m r_{jm},$$

where ω_j captures the importance of decision node j , and λ_m captures the importance of right type m . In low-risk drafting tasks, reasoning and output-generation rights may matter more than final validation. In regulated credit, medical, cybersecurity, or infrastructure settings, action-selection, validation, and memory-update rights may receive higher weights.

The purpose of the aggregate profile is threefold. First, it provides a compact diagnostic: a system may be globally LLM-dominant even if it has a few human approval gates. Second, it allows high-level comparison across systems after their processes are aligned to a common node ontology. Third, it can be used inside an optimization model when the full matrix is too detailed for early-stage design. However, the aggregate should always be traceable back to the underlying node-level rights because two systems with the same R may allocate rights very differently across high-risk and low-risk nodes.

Comparison does not require two systems to have identical components. It requires a mapping between

decision nodes or task functions. For example, two credit-decision systems can be compared by aligning intake, evidence retrieval, eligibility assessment, explanation drafting, final approval, and record update, even if one uses RAG and the other uses a rules database. After such alignment, a distance between two designs can be represented as

$$\Delta(A, B) = \sum_{j=1}^J \sum_{m=1}^M \omega_j \lambda_m \|r_{jm}^A - r_{jm}^B\|_1.$$

The node weights ω_j and right-type weights λ_m can be set in several ways. In a qualitative paper, equal weights may be sufficient for exposition. In an applied system, ω_j can be estimated from decision frequency, consequence severity, reversibility, regulatory exposure, and historical failure cost. The weights λ_m can be elicited from domain experts, compliance requirements, user-impact analysis, or structured methods such as pairwise comparison. They can also be learned or adjusted from incident logs if the organization has enough operational data. The important point is that weights should represent the local value and risk of rights, not the salience of a component in the architecture diagram.

Extreme points and trade-offs

The three extreme points of the allocation space illustrate the trade-offs. Let

$$e_H = (1, 0, 0), \quad e_D = (0, 1, 0), \quad e_L = (0, 0, 1),$$

where e_H is a human-dominant allocation, e_D is a deterministic-dominant allocation, and e_L is an LLM-dominant allocation for a given node-right pair. A concrete design usually chooses a point inside the simplex rather than a vertex. For a local task condition vector x_{jm} , the desired allocation can be represented abstractly as

$$r_{jm}^* \in \arg \max_{r \in \Delta^2} B_{jm}(r; x_{jm}) - C_{jm}(r; x_{jm}),$$

where B_{jm} captures the expected benefits of allocating that right to humans, deterministic components, and the LLM, and C_{jm} captures the corresponding costs and risks. The following three extreme cases should therefore be read as illustrative anchors for the later optimization model, not as recommended system-wide designs.

At the **human-dominant** extreme, humans hold most decision rights. This is often appropriate when the task is rare, high-stakes, normatively sensitive, or heavily dependent on tacit expertise. It maximizes human accountability and contextual judgment, but it limits scalability and responsiveness. Human-dependent anomaly-management pipelines, for example, can constrain responsiveness when timely intervention is required in complex systems (Barenji and Khoshgoftar 2025).

At the **deterministic-dominant** extreme, rules, schemas, process engines, retrievers, validators, and formal knowledge structures hold most decision rights. This can improve reliability, auditability, replayability, and

cost control. It is appropriate when the task is stable, codifiable, and measurable. Deterministic validation can also expose unsupported factual claims by translating claims into symbolic checks or knowledge-base queries (Vakharia et al. 2024), and neuro-symbolic pipelines can move parts of reasoning into explicit rules, ontologies, or automated consistency checks (Vsevolodovna and Monti 2025; Garrido-Merchán and Puente 2025). However, deterministic systems can be brittle when behavior evolves, novel-but-valid patterns appear, or sensor noise creates false positives (Barenji and Khoshgoftar 2025). They may also be difficult to build when expert knowledge is tacit or when the relevant context changes faster than rules can be maintained.

At the **LLM-dominant** extreme, the LLM holds broad rights over interpretation, reasoning, planning, and action selection. This can improve adaptability, natural-language usability, and coverage across heterogeneous cases. It is attractive for tasks that are difficult to specify in advance. But it can reduce auditability, increase hallucination risk, create integration errors, and produce outputs that are cheap to generate but expensive to verify. The empirical results of Z. Z. Wang et al. (2025) suggest that LLM agents may be fast and cheap while still producing lower-quality work in many realistic tasks. Repetitive deterministic-prediction tasks may also reveal sequence-level accuracy cliffs as output length grows (Hou et al. 2025), reinforcing the point that LLM-dominant rights should be matched to tasks where semantic flexibility is more valuable than exact repeatability.

Most useful agentic systems are therefore hybrid. The design question is not whether an LLM should be present, but which rights it should hold, which rights should be constrained by deterministic infrastructure, and which rights should remain with humans.

A model for choosing the desirable allocation

The decision-rights perspective becomes useful when it guides design choices. We propose a simple comparative model. A system design a implies a node-by-right matrix $\{r_{jm}(a)\}_{j,m}$ and an aggregate rights profile

$$R(a) = (H(a), D(a), L(a)), \quad H(a) + D(a) + L(a) = 1.$$

The desirable design maximizes expected system value subject to the rights matrix induced by the architecture, interface, model choice, process policy, and oversight design:

$$\begin{aligned} a^* &= \arg \max_a \mathbb{E}[V(a)] \\ &= B_{\text{scale}}(a) + B_{\text{adapt}}(a) + B_{\text{quality}}(a) \\ &\quad - C_{\text{human}}(a) - C_{\text{compute}}(a) - C_{\text{integration}}(a) - C_{\text{risk}}(a) - C_{\text{audit}}(a) - C_{\text{error_scale}}(a). \end{aligned}$$

This formulation is deliberately high-level. It does not assume a universal linear function. Instead, it identifies variables that should change the desirable allocation. The additional term $C_{\text{error_scale}}(a)$ is not presented as a distinctive feature of LLMs alone. Rather, it captures a deployment consequence of scalable automation: when stochastic outputs are connected to tools, memory, and external processes, even a small

error rate can create large aggregate review, remediation, compliance, or user-harm costs.

Variable	Expected effect on LLM rights	Expected effect on deterministic rights	Expected effect on human rights
Scalability pressure	Increases, if quality and safeguards are acceptable	Increases when tasks are codifiable or checkable	Decreases when human review is bottleneck
Need for adaptability	Increases for open-ended interpretation and planning	Decreases if rules cannot keep up	Increases when adaptation requires tacit judgment
Deterministic codifiability	Decreases	Increases	Decreases after expert knowledge is codified
Task risk and irreversibility	Decreases unless strong safeguards exist	Increases through constraints and validators	Increases through approval and accountability
Auditability and regulatory burden	Decreases for opaque reasoning	Increases through logs, schemas, rules, and provenance	Increases through review and sign-off
LLM task reliability	Increases	May decrease for that subtask	May decrease for routine validation
Verification availability	Increases when outputs can be automatically checked	Increases through validators and tests	Decreases for routine checks but remains for exceptions
Human capacity or cost pressure	Increases for low-risk tasks	Increases for automatable subtasks	Decreases except for escalation
Tacitness of domain knowledge	Increases for elicitation and drafting	Increases after knowledge is codified	Increases for validation and correction
Security and prompt-injection exposure	Decreases for untrusted inputs	Increases through filtering, sandboxing, and least-privilege tools	Increases for high-risk exceptions
Need for user feedback and contestability	Increases for feedback summarization and triage	Increases through feedback schemas and issue trackers	Increases for appeal, correction, and policy revision

The model yields several design propositions.

Proposition 1: LLM decision rights should increase with adaptability needs, but only when verification is available. LLMs are valuable when tasks require semantic flexibility, novel interpretation, or rapid response to changing environments. This is why rule-based anomaly management faces limitations in evolving systems (Barenji and Khoshgoftar 2025). However, adaptability without verification can produce fabricated or misdirected actions. Therefore, high LLM rights are most defensible when paired with deterministic tests, provenance, rollback, human escalation, or other validation mechanisms.

Proposition 2: deterministic decision rights should increase with codifiability, risk, and auditability requirements. When a decision can be represented as a rule, schema, ontology, retrieval policy, router, or validator, deterministic infrastructure can reduce variance and improve auditability. This does not require removing the LLM. The LLM can still interpret inputs or draft outputs, while deterministic components hold rights over admissible evidence, valid actions, arithmetic, compliance checks, routing thresholds, and memory write conditions.

Proposition 3: human decision rights should concentrate at high-leverage points rather than appear everywhere. Human-in-the-loop design is often necessary, but indiscriminate human review weakens scalability and may create rubber-stamping. A better design places humans where their rights are most valuable: goal framing, validation of high-risk actions, adjudication of ambiguous cases, approval of persistent memory, review of user feedback, and refinement of domain protocols. This interpretation is consistent with human-centered evaluation in optimization workflows, where interpretability, computational efficiency, domain constraints, and tacit preferences are hard to encode in fixed fitness functions (W. Li et al. 2025).

Proposition 4: memory-update rights require stricter governance than one-shot generation rights. A one-time LLM error can be corrected. A stored error can become a future premise. Memory systems therefore require explicit rights over writing, retrieval, versioning, provenance, and deletion. Persistent reasoning reuse should be conditioned on policy constraints such as acyclicity, type compatibility, semantic thresholds, version control, and provenance metadata (Singh 2025).

Proposition 5: the optimal allocation is node-specific, not system-wide. A single system may give the LLM high rights in drafting, deterministic components high rights in retrieval and validation, and humans high rights in approval and escalation. Treating an entire system as either autonomous or human-controlled obscures these internal allocations.

Proposition 6: high scalability raises both the value of delegation and the value of control. When a task occurs many times, delegating rights to LLMs or deterministic systems can create large productivity gains. But the same frequency increases the expected cost of systematic errors, biased defaults, memory contamination, and user-harm amplification. Therefore, scalable systems should not simply move rights from humans to LLMs; they should move routine, checkable rights to deterministic components and reserve human attention for high-leverage exceptions.

Design patterns as decision-right-shifting mechanisms

Common agentic-system design patterns can be reinterpreted as mechanisms for reallocating decision rights. The model above makes this interpretation explicit. Let a design pattern or architectural intervention be denoted by g . Applying g to a baseline design a changes the rights matrix by

$$\delta_{jm}(g \mid a) = r_{jm}(a \circ g) - r_{jm}(a),$$

and should be adopted only when its expected value gain is positive relative to feasible alternatives:

$$\Delta V(g \mid a) = \mathbb{E}[V(a \circ g)] - \mathbb{E}[V(a)] > 0.$$

Figure 2 illustrates the matrix which maps six process nodes to six decision-right categories and shows the relative allocation of authority among human, deterministic, and LLM components in each cell.

Node-by-right decision-rights matrix

	Goal framing	Information selection	Reasoning	Action selection	Validation	Memory update
Goal intake						
Evidence retrieval						
Evidence interpretation						
Tool/action choice						
Output validation						
Memory write						

Legend

H = Human D = Deterministic L = LLM

The aggregate system profile summarizes this matrix, but should remain traceable to node-level allocations.

Figure 2: Node-by-right decision-rights matrix.

Thus, a pattern is not intrinsically good. RAG, memory, routing, HITL, schema validation, workflow reuse, and multi-agent orchestration are valuable only when the rights they shift improve the local trade-off between adaptability, scalability, quality, verification, auditability, cost, and risk. This section maps several patterns to the rights they primarily affect and connects the six-right framework to related classifications in the literature.

Design pattern	Primary mechanism	Main decision-right shift	Important caveat
Retrieval-augmented generation (RAG)	Retrieves external documents or records before generation	Shifts information-selection rights from the LLM’s parametric memory toward retrieval infrastructure	If the LLM writes the query, chooses sources, or resolves conflicts without validation, substantial rights remain with the LLM

Design pattern	Primary mechanism	Main decision-right shift	Important caveat
Episodic and cross-workflow memory	Stores and retrieves prior interactions, process traces, reasoning traces, or reusable workflow segments	Shifts information-selection, reasoning, and memory-update rights toward memory infrastructure	If the LLM writes memories without approval, errors can become persistent precedents
Human-in-the-loop or human-on-the-loop validation	Requires human approval, correction, monitoring, or escalation	Shifts validation and sometimes action-selection rights to humans	Can create bottlenecks; oversight may be superficial if upstream framing is controlled by the LLM
Domain-expert knowledge codification	Converts expert heuristics into protocols, rules, examples, or executable guidance	Shifts recurring reasoning and validation rights from humans into deterministic guidance that constrains the LLM	Codified knowledge can become stale; tacit knowledge may be only partially captured
Provenance tracing and audit logs	Records evidence, intermediate states, tool calls, and decision paths	Strengthens ex post human and deterministic validation rights	Provenance does not itself prevent bad decisions unless linked to validation or rollback mechanisms
Reinforcement learning and feedback adaptation	Updates future behavior according to rewards or feedback	Shifts future reasoning, action-selection, and memory-update rights toward the learned policy and reward specification	Poor reward design can over-optimize measurable outcomes while weakening accountability
Ontology-derived semantic context	Uses formal concepts and relationships to structure interpretation	Shifts information-selection and reasoning rights toward deterministic semantic models	Ontologies require maintenance and human validation to avoid institutional amnesia or false structure
Routers and cascades	Selects models, prompts, retrieval methods, precision levels, or tools based on task features	Shifts action-selection rights from ad hoc prompting to deterministic or learned routing	Router errors can silently send tasks to inappropriate components

Design pattern	Primary mechanism	Main decision-right shift	Important caveat
Schema and contract validation	Requires structured, valid, policy-compliant outputs before execution	Shifts validation and action-selection rights to deterministic interfaces	Valid format does not guarantee correct reasoning or appropriate action
Multi-agent orchestration	Decomposes tasks among specialized agents or roles	Distributes goal-framing, reasoning, action-selection, and validation rights across agents, supervisors, tools, and humans	Without explicit governance, distributed autonomy can obscure responsibility
Agent evaluation and judging	Evaluates outputs, trajectories, artifacts, or intermediate workspaces	Shifts validation rights toward human judges, LLM judges, agent judges, or deterministic test suites	Evaluation rights can be biased or incomplete if judges see only final outputs

This table complements typological work on agentiveness. Wissuchek and Zschech (2025) classifies systems by dimensions of cognitive and environmental agency. The table above asks a different question: once a system has a given capability, which party controls that capability in deployment? It also complements decoupled HITL architectures, which treat human approval, escalation, and feedback as an independent component of the agent operating environment (Cheng and Cheng 2026). In rights terms, a decoupled HITL layer is valuable because it makes validation and escalation rights explicit rather than embedding them informally inside each application.

RAG is a simple example. In a purely generative system, the LLM has broad rights over what information it uses and how it justifies an answer. In a RAG system, a retriever can narrow the evidence set and thereby reduce the LLM’s right to invent or select facts. The shift is incomplete, however, if the LLM controls query formulation, source ranking, or conflict resolution. In matrix terms, RAG should reduce l_{jm} and increase d_{jm} for information-selection rights at the relevant evidence node. If the LLM controls retrieval and conflict resolution, l_{jm} remains high, so the design may not reduce C_{risk} or C_{audit} as much as the component label suggests. This explains why RAG design is not only about accuracy; it is about allocating evidence-selection rights.

Memory and workflow reuse have a similar structure. Fiorini et al. (2025) describes provenance-backed workflow databases as a basis for episodic memory in agentic workflows. Singh (2025) describes graph-memoized reasoning as persistent graph-structured memory that allows prior reasoning subgraphs to be retrieved and recomposed. Tang et al. (2025) extends the idea to cross-framework agent experience, while Gaoyang Zhang et al. (2026) treats validated workflow segments as reusable enterprise assets. These designs can reduce repeated reasoning and improve adaptation, but they also create governance questions: who decides which traces are stored, which traces are retrieved, which workflow segment is reusable, and when

a prior trace is no longer valid? In the model, memory can increase B_{adapt} and reduce $C_{compute}$ through reuse, but it can also increase C_{risk} if δ_{jm} gives the LLM high memory-update rights without validation.

Human-in-the-loop validation is the most explicit rights-shifting pattern. It gives humans approval, correction, or veto rights. Many applied systems use this pattern because full LLM autonomy remains unreliable in high-stakes or feedback-sensitive tasks. Human oversight appears in software testing (Naqvi, Baqar, and Mohammad 2026; Hariharan 2025), education (Yu et al. 2025), and rice-yield recommendation (Yamaguchi et al. 2025). Yet human-in-the-loop design can be weaker than it appears if the human sees only the final answer and not the evidence, tool calls, rejected alternatives, or uncertainty. Effective human rights require observability and meaningful intervention points. In matrix terms, HITL should increase h_{jm} for validation rights, but it may leave l_{jm} high for information-selection and reasoning rights unless the interface exposes those upstream decisions.

Domain-expert knowledge codification shifts rights in a different direction. Uulu et al. (2026) argues that codified domain-expert knowledge can improve agent outputs beyond RAG-only code generation, because factual retrieval alone may not encode what makes an output analytically useful. Guangwei Zhang (2025) similarly argues that knowledge protocols can specialize general-purpose models through workflows, decision trees, logical dependencies, and heuristic strategies. L. Zhang (2026) adds that domain-agent development may require continuous practitioner interaction because domain expertise can be tacit, personal, and evolving. In decision-right terms, these methods move repeated expert judgments from direct human intervention into deterministic or semi-deterministic guidance, while preserving human rights over correction and recodification when the domain changes. In the model, this can reduce C_{human} while increasing $D(a)$ for recurring reasoning and validation rights; however, it remains desirable only while the codified knowledge is maintained and contestable.

Provenance tracing, audit logs, and justification loops are often described as explainability tools, but their governance function is broader. They create the conditions for review, contestation, and accountability. McGee et al. (2025) proposes a Toulmin-style justification loop for inspecting agentic decisions through explicit claims, evidence, warrants, backing, rebuttals, and qualifiers. J. Zhu et al. (2026) argues for system-level interpretability because agentic behavior emerges across modules. Ghosh et al. (2025) similarly treats agentic risk as a system-level property that can propagate across tool calls, memory, RAG, APIs, and orchestration. These mechanisms do not necessarily reduce LLM decision rights at the moment of generation, but they increase human and deterministic rights to audit, reject, revise, and learn from decisions afterward. Their value is therefore largest when C_{audit} , regulatory exposure, or downstream reversibility costs are high.

Reinforcement learning and feedback adaptation allocate rights over future behavior. In software testing, Hariharan (2025) describes reward dimensions such as effectiveness, coverage, efficiency, compliance, and adaptation. In interaction learning, Klissarov et al. (2026) argues that feedback adaptation should be optimized rather than left to prompt engineering. In decision-right terms, reward functions and feedback mechanisms decide which behaviors become more likely. This shifts some rights from current human operators or prompt designers into the learned policy. The benefits are adaptability and continual improvement; the risks are reward misspecification, hidden drift, and reduced interpretability. Matrix analysis clarifies that feedback adaptation changes not only present performance but future r_{jm} values.

Ontology-derived semantic context shifts representational rights, which are treated here as a subcase of information-selection and reasoning rights. McGee et al. (2025) argues that ontology-derived semantic context can improve LLM response accuracy and coherence in enterprise operations and that human validation of AI-generated knowledge structures can reduce institutional amnesia. The ontology constrains what concepts and relations the system uses, while human validation prevents the ontology from becoming a misleading deterministic artifact. This pattern is especially important when organizations want agents to operate over institutional knowledge rather than over free-form retrieved text.

Routers and cascades allocate rights over component selection. Routing can choose among LLMs, quantized and full-precision variants, tools, direct answering, retrieval, or human escalation. Learned routing work shows that routing can be optimized for quality, cost, latency, and decision regret (Sikeridis, Ramdass, and Pareek 2024; Tsiourvas, Sun, and Perakis 2025; Qian et al. 2025; Y. Li et al. 2026). It can also fail through routing collapse, where a policy increasingly defaults to the strongest and most expensive model even when cheaper models are sufficient (Lai and Ye 2026). In decision-right terms, routers are governance mechanisms because they decide which party or component will exercise downstream reasoning and action-selection rights. A router is desirable when its δ_{jm} lowers expected cost without silently moving high-risk reasoning rights to weak or poorly audited components.

Multi-agent orchestration distributes rights across multiple LLMs, tools, and humans. Frameworks such as AutoGen demonstrate how agent interactions can be programmed through conversations, Python control logic, auto-reply functions, and group chats (Wu et al. 2023). Framework-agnostic representations such as Agent Spec attempt to standardize agent components, control-flow semantics, data-flow semantics, and runtime adapters (Amini et al. 2025). These efforts are important because rights become harder to see when they are spread across agents. A rights-aware multi-agent design should specify not only roles, but also which agent can set goals, select evidence, call tools, approve actions, and write memory.

Illustrative design implications

Autonomous anomaly management

Autonomous anomaly management illustrates the value of increasing LLM rights under scalability and adaptability pressure. Human-dependent anomaly-management pipelines can be too slow for complex systems, while rule-based approaches may struggle when system behavior evolves (Barenji and Khoshgoftar 2025). An LLM-enabled agent can interpret heterogeneous logs, sensor descriptions, and operational context, then propose explanations and responses. Yet full LLM control would be risky because anomaly response may affect real infrastructure. A rights-aware design would allocate information synthesis and hypothesis generation partly to the LLM, anomaly thresholds and policy constraints to deterministic components, and high-impact response approval to humans. As the system matures, repeated expert responses could be codified into deterministic playbooks, gradually shifting recurring rights away from humans without granting unconstrained authority to the LLM.

Business-process management and process governance

Business-process management illustrates what changes when systems evolve from predefined automation toward more autonomous process performers. Dumas, Milani, and Chapela-Campa (2026) argues that agentic AI shifts BPM from automation toward autonomy and from design-driven process management toward data-driven process management supported by process mining. In the decision-rights view, this transition is not merely a new process technology. It is a redistribution of authority from predefined process logic and human process owners toward agents that can sense process states, reason about improvement opportunities, and act to maintain or optimize performance. Because BPM systems operate inside organizational processes, the key implication is that autonomy must remain framed by verification-centric process design: deterministic process constraints, provenance, role definitions, escalation rules, and human accountability should specify which rights agents receive and which rights remain with process owners.

Workplace task automation

The workplace-agent evidence in Z. Z. Wang et al. (2025) suggests that current agents can be fast and inexpensive but still flawed. A rights-aware interpretation is that agents should not automatically receive end-to-end task rights merely because they can operate software tools. Instead, programmable subtasks with clear verification can be delegated to LLM or deterministic components, while visually grounded, ambiguous, or quality-sensitive steps should retain human validation. This supports augmentation over full automation when the LLM's process is efficient but its output quality remains uncertain. It also suggests that process demonstrations and induced routines can be used to identify which decision nodes are safe to delegate.

Education and feedback-sensitive work

Educational AI illustrates the importance of human validation and contextual judgment. AI grading or feedback tools can improve efficiency, but rigid answer-key matching and automated feedback can miss contextually correct student responses (Yu et al. 2025). A rights-aware design would give the LLM drafting rights for feedback, deterministic components rights over rubric structure and consistency checks, instructors validation rights over grades or sensitive interventions, and students or end users contestation rights when feedback seems inappropriate. Over time, instructor corrections and student feedback can be converted into evaluation rules or examples, shifting some repeated judgments into deterministic or retrieval-based guidance while preserving human authority over ambiguous cases.

Domain-expert visualization and engineering workflows

In engineering or visualization tasks, RAG alone may retrieve relevant information but fail to produce expert-quality outputs. Uulu et al. (2026) argues that codified domain-expert knowledge can improve visualization generation beyond RAG-only code generation. Under the decision-rights view, the key mechanism is that expert codification shifts rights over quality criteria, chart selection, and domain appropriateness away from the LLM's generic judgment and into reusable protocols. The LLM still performs natural-language interpretation and code generation, but deterministic or expert-authored guidance controls what counts as an acceptable solution.

Model routing and cost-sensitive orchestration

Model routing illustrates that decision rights also apply to component selection. A system may use a small model for routine classification, a stronger model for difficult reasoning, a deterministic calculator for arithmetic, a retrieval pipeline for evidence, and a human reviewer for sensitive exceptions. Routing research shows that model selection can be optimized for quality, latency, cost, and regret (Sikeridis, Ramdass, and Pareek 2024; Tsiourvas, Sun, and Perakis 2025; Lai and Ye 2026). A rights-aware interpretation is that the router holds action-selection rights over which component will decide next. Therefore, router governance matters: a poor router can silently give weak models high-stakes reasoning rights or overuse expensive models in cases where deterministic checks would suffice.

Discussion

Decision-rights allocation helps explain why apparently similar agentic systems differ in reliability and governance. Two systems may both use RAG, memory, and tool calling, but one may let the LLM control query generation, source selection, memory writing, and action execution, while another uses curated retrieval, validated memory, schema-constrained tool calls, and human approval. Component labels alone cannot capture this difference because they identify the presence of a mechanism, not the authority distribution inside that mechanism. Decision-rights allocation adds this missing layer.

The perspective also helps unify several debates. The debate between rules and LLMs becomes a question of when deterministic rights should constrain stochastic reasoning. The debate between automation and augmentation becomes a question of which decision nodes should retain human rights. The debate about autonomy becomes a question of which rights are transferred away from humans, not simply how independently the system operates. The debate about explainability becomes a question of whether humans and deterministic systems have enough information to exercise review and correction rights. The debate about agent memory becomes a question of who controls persistence and reuse.

This paper does not claim that decision rights are easy to measure. Effective rights may differ from formal rights. A human may formally approve an output but lack the information needed to challenge it. A deterministic validator may formally constrain action format but fail to constrain action meaning. An LLM may formally only draft recommendations but effectively determine the decision by framing the options. Future work should therefore develop empirical methods for estimating effective rights from system logs, interface design, approval behavior, routing records, memory-write events, user feedback, and failure cases. Agent evaluation should also move beyond final-output scoring toward trajectory, workspace, artifact, safety, robustness, cost, and long-horizon assessment (Yehudai et al. 2025; Zhuge et al. 2024).

Another limitation is that the three-party model is intentionally coarse. It treats all humans as one party, but organizations contain users, developers, managers, auditors, and affected communities with different interests. It treats deterministic components as one party, but rule engines, retrievers, validators, and routers can conflict. It treats stochastic LLMs as one party, but multi-agent systems may contain multiple models with different roles. The coarse model is useful for a first draft because it clarifies the main trade-off, but later versions should extend it to multi-stakeholder and multi-agent settings.

A further limitation is the boundary of the LLM component. In open-source deployments, designers may inspect prompts, retrieval, decoding settings, validators, and model versions. In API-based deployments, some provider-side safety filters, retrieval, or tool-use mechanisms may be hidden. The framework handles this by distinguishing explicit rights from observed reliability: hidden provider mechanisms may make a service safer, but they do not give the deploying organization explicit deterministic control rights unless they are exposed through inspectable interfaces. This boundary issue should become an empirical research topic because many organizations will govern agentic systems through third-party model services rather than fully transparent model stacks.

Conclusion

LLM-enabled agentic systems are not only collections of models, tools, memories, and workflows. They are arrangements of decision rights. Every design choice determines whether humans, deterministic components, or stochastic LLMs have authority over goals, evidence, reasoning, actions, validation, and memory. This paper proposed decision-rights allocation as a compact internal-mechanism perspective for comparing and designing such systems. The perspective is needed because LLM-enabled agents place open-ended semantic judgment inside operational control loops, where generated interpretations can shape evidence, actions, and future state. The paper showed how common design patterns can be interpreted as rights-shifting mechanisms and proposed a simple model for choosing the desirable allocation based on scalability, adaptability, risk, auditability, codifiability, model reliability, verification availability, human capacity, and the cost of errors at scale.

The main practical implication is that agentic-system design should not begin with the question, “How autonomous can the agent be?” It should begin with the question, “Which party should hold which decision rights at which process step?” LLMs should receive rights where semantic flexibility and adaptation create value; deterministic components should receive rights where rules, schemas, retrieval, validators, provenance, and routing improve reliability; humans should retain rights where judgment, accountability, feedback, and legitimacy matter. A theory of LLM-enabled agentic systems can then be built not around autonomy as a single aspiration, but around explicit, inspectable, and optimizable allocation of decision rights.

References

- Amini, Soufiane, Yassine Benajiba, Cesare Bernardis, Paul Cayet, Hassan Chafi, Abderrahim Fathan, Louis Faucon, et al. 2025. “Open Agent Specification (Agent Spec): A Unified Representation for AI Agents.” <https://arxiv.org/abs/2510.04173v4>.
- Barenji, Reza Vatankhah, and Sina Khoshgoftar. 2025. “Agentic AI for Autonomous Anomaly Management in Complex Systems.” <https://arxiv.org/abs/2507.15676v1>.
- Cao, Sheng, Zhao Chang, Chang Li, Hannan Li, Liyao Fu, and Ji Tang. 2026. “The Auton Agentic AI Framework.” <https://arxiv.org/abs/2602.23720v1>.
- Cheng, Edward, and Jeshua Cheng. 2026. “A Decoupled Human-in-the-Loop System for Controlled Autonomy in Agentic Workflows.” <https://arxiv.org/abs/2604.23049v1>.

- Cihon, Peter, Merlin Stein, Gagan Bansal, Sam Manning, and Kevin Xu. 2025. "Measuring AI Agent Autonomy: Towards a Scalable Approach with Code Inspection." <https://arxiv.org/abs/2502.15212v1>.
- Dumas, Marlon, Fredrik Milani, and David Chapela-Campa. 2026. "Agentic Business Process Management Systems." <https://arxiv.org/abs/2601.18833v1>.
- Feng, K. J. Kevin, David W. McDonald, and Amy X. Zhang. 2025. "Levels of Autonomy for AI Agents." <https://arxiv.org/abs/2506.12469v2>.
- Fiorini, Sandro Rama, Leonardo G. Azevedo, Raphael M. Thiago, Valesca M. de Sousa, Anton B. Labate, and Viviane Torres da Silva. 2025. "Episodic Memory in Agentic Frameworks: Suggesting Next Tasks." <https://arxiv.org/abs/2511.17775v1>.
- Garrido-Merchán, Eduardo C., and Cristina Puente. 2025. "GOF AI Meets Generative AI: Development of Expert Systems by Means of Large Language Models." <https://arxiv.org/abs/2507.13550v2>.
- Ghosh, Shaona, Barnaby Simkin, Kyriacos Shiarlis, Soumili Nandi, Dan Zhao, Matthew Fiedler, Julia Bazinska, et al. 2025. "A Safety and Security Framework for Real-World Agentic Systems." <https://arxiv.org/abs/2511.21990v1>.
- Gumaan, Esmail. 2025. "Theoretical Foundations and Mitigation of Hallucination in Large Language Models." <https://arxiv.org/abs/2507.22915v1>.
- Hariharan, Mohanakrishnan. 2025. "Reinforcement Learning Integrated Agentic RAG for Software Test Cases Authoring." <https://arxiv.org/abs/2512.06060v1>.
- Hou, Wanda, Leon Zhou, Hong-Ye Hu, Yubei Chen, Yi-Zhuang You, and Xiao-Liang Qi. 2025. "How Focused Are LLMs? A Quantitative Study via Repetitive Deterministic Prediction Tasks." <https://arxiv.org/abs/2511.00763v2>.
- Hu, Shengran, Cong Lu, and Jeff Clune. 2024. "Automated Design of Agentic Systems." <https://arxiv.org/abs/2408.08435v2>.
- Joshi, Tanmay, Shourya Aggarwal, Anusa Saha, Aadi Pandey, Shreyash Dhoot, Vighnesh Rai, Raxit Goswami, Aman Chadha, Vinija Jain, and Amitava Das. 2026. "Stochastic CHAOS: Why Deterministic Inference Kills, and Distributional Variability Is the Heartbeat of Artificial Cognition." <https://arxiv.org/abs/2601.07239v1>.
- Kamalov, Firuz, David Santandreu Calonge, Linda Smail, Dilshod Azizov, Dimple R. Thadani, Theresa Kwong, and Amara Atif. 2025. "Evolution of AI in Education: Agentic Workflows." <https://arxiv.org/abs/2504.20082v2>.
- Kandikatla, Laxmiraju, and Branislav Radeljic. 2025. "AI and Human Oversight: A Risk-Based Framework for Alignment." <https://arxiv.org/abs/2510.09090v1>.
- Kiprono, Moses. 2025. "Mathematical Analysis of Hallucination Dynamics in Large Language Models: Uncertainty Quantification, Advanced Decoding, and Principled Mitigation." <https://arxiv.org/abs/2511.15005v1>.
- Klissarov, Martin, Jonathan Cook, Diego Antognini, Hao Sun, Jingling Li, Natasha Jaques, Claudiu Musat, and Edward Grefenstette. 2026. "Improving Interactive in-Context Learning from Natural Language Feedback." <https://arxiv.org/abs/2602.16066v1>.
- Kubam, Chandra Sekhar. 2025. "Agentic AI for Autonomous, Explainable, and Real-Time Credit Risk Decision-Making." <https://doi.org/10.5281/zenodo.18008872>.
- Lai, Guannan, and Han-Jia Ye. 2026. "When Routing Collapses: On the Degenerate Convergence of LLM

- Routers.” <https://arxiv.org/abs/2602.03478v1>.
- Laird, John E. 2022. “Introduction to Soar.” <https://arxiv.org/abs/2205.03854v1>.
- Lamo Castrillo, Victor de, Habtom Kahsay Gidey, Alexander Lenz, and Alois Knoll. 2025. “Fundamentals of Building Autonomous LLM Agents.” <https://arxiv.org/abs/2510.09244v1>.
- Lazer, Sahaya Jestus, Kshitiz Aryal, Maanak Gupta, and Elisa Bertino. 2026. “A Survey of Agentic AI and Cybersecurity: Challenges, Opportunities and Use-Case Prototypes.” <https://arxiv.org/abs/2601.05293v1>.
- Li, Wenhao, Bo Jin, Mingyi Hong, Changhong Lu, and Xiangfeng Wang. 2025. “Optimization Problem Solving Can Transition to Evolutionary Agentic Workflows.” <https://arxiv.org/abs/2505.04354v1>.
- Li, Yuanzhe, Jianing Deng, Jingtong Hu, Tianlong Chen, Song Wang, and Huanrui Yang. 2026. “Dynamic Mix Precision Routing for Efficient Multi-Step LLM Interaction.” <https://arxiv.org/abs/2602.02711v1>.
- McGee, Liam, James Harvey, Lucy Cull, Andreas Vermeulen, Bart-Floris Visscher, and Malvika Sharan. 2025. “Enabling Ethical AI: A Case Study in Using Ontological Context for Justified Agentic AI Decisions.” <https://arxiv.org/abs/2512.04822v1>.
- Milosevic, Zoran, and Fethi Rabhi. 2026. “Architecting Agentic Communities Using Design Patterns.” <https://arxiv.org/abs/2601.03624v2>.
- Naqvi, Saba, Mohammad Baqar, and Nawaz Ali Mohammad. 2026. “The Rise of Agentic Testing: Multi-Agent Systems for Robust Software Quality Assurance.” <https://arxiv.org/abs/2601.02454v1>.
- Pellejero, Jorge, Luis A. Hernández Gómez, Luis Mendo Tomás, and Zoraida Frias Barroso. 2025. “Agentic AI for Mobile Network RAN Management and Optimization.” <https://arxiv.org/abs/2511.02532v1>.
- Pittman, Jason M. 2024. “A Measure for Level of Autonomy Based on Observable System Behavior.” <https://arxiv.org/abs/2407.14975v1>.
- Qian, Cheng, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jielin Qiu, Haolin Chen, Zhiwei Liu, et al. 2025. “xRouter: Training Cost-Aware LLMs Orchestration System via Reinforcement Learning.” <https://arxiv.org/abs/2510.08439v1>.
- Sapkota, Ranjan, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. “AI Agents Vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges.” <https://doi.org/10.1016/j.inffus.2025.103599>.
- Sikeridis, Dimitrios, Dennis Ramdass, and Pranay Pareek. 2024. “PickLLM: Context-Aware RL-Assisted Large Language Model Routing.” <https://arxiv.org/abs/2412.12170v1>.
- Singh, Yash Raj. 2025. “Graph-Memoized Reasoning: Foundations Structured Workflow Reuse in Intelligent Systems.” <https://arxiv.org/abs/2511.15715v1>.
- Son, Tong Duy, Zhihao Liu, Piero Brigida, Yerlan Akhmetov, Gurudevan Devarajan, Kai Liu, and Ajinkya Bhawe. 2026. “Automotive Engineering-Centric Agentic AI Workflow Framework.” <https://arxiv.org/abs/2604.07784v1>.
- Tang, Xiangru, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, et al. 2025. “Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving.” <https://arxiv.org/abs/2507.06229v5>.
- Taparia, Aditya, Ransalu Senanayake, Kowshik Thopalli, and Vivek Narayanaswamy. 2026. “The Anatomy of Uncertainty in LLMs.” <https://arxiv.org/abs/2603.24967v1>.
- Tsiourvas, Asterios, Wei Sun, and Georgia Perakis. 2025. “Causal LLM Routing: End-to-End Regret Minimization from Observational Data.” <https://arxiv.org/abs/2505.16037v2>.

- Uulu, Choro Ulan, Mikhail Kulyabin, Iris Fuhrmann, Jan Joosten, Nuno Miguel Martins Pacheco, Filippos Petridis, Rebecca Johnson, Jan Bosch, and Helena Holmström Olsson. 2026. “How to Build AI Agents by Augmenting LLMs with Codified Human Expert Domain Knowledge? A Software Engineering Framework.” <https://arxiv.org/abs/2601.15153v1>.
- Vakharia, Priyesh, Abigail Kufeldt, Max Meyers, Ian Lane, and Leilani Gilpin. 2024. “ProSLM : A Prolog Synergized Language Model for Explainable Domain Specific Knowledge Based Question Answering.” https://doi.org/10.1007/978-3-031-71170-1_23.
- Vsevolodovna, Ruslan Idelfonso Magana, and Marco Monti. 2025. “Enhancing Large Language Models Through Neuro-Symbolic Integration and Ontological Reasoning.” <https://arxiv.org/abs/2504.07640v2>.
- Wang, Jun. 2025. “Memento 2: Learning by Stateful Reflective Memory.” <https://arxiv.org/abs/2512.22716v3>.
- Wang, Lei, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, et al. 2023. “A Survey on Large Language Model Based Autonomous Agents.” <https://doi.org/10.1007/s11704-024-40231-1>.
- Wang, Zora Zhiruo, Yijia Shao, Omar Shaikh, Daniel Fried, Graham Neubig, and Diyi Yang. 2025. “How Do AI Agents Do Human Work? Comparing AI and Human Workflows Across Diverse Occupations.” <https://arxiv.org/abs/2510.22780v2>.
- Wissuchek, Christopher, and Patrick Zschech. 2025. “Exploring Agentic Artificial Intelligence Systems: Towards a Typological Framework.” <https://arxiv.org/abs/2508.00844v1>.
- Wray, Robert E., James R. Kirk, and John E. Laird. 2025. “Applying Cognitive Design Patterns to General LLM Agents.” <https://arxiv.org/abs/2505.07087v2>.
- Wu, Qingyun, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, et al. 2023. “Auto-Gen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” <https://arxiv.org/abs/2308.08155v2>.
- Yamaguchi, Tomoaki, Yutong Zhou, Masahiro Ryo, and Keisuke Katsura. 2025. “Agentic Explainable Artificial Intelligence (Agentic XAI) Approach to Explore Better Explanation.” <https://arxiv.org/abs/2512.21066v3>.
- Yehudai, Asaf, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. “Survey on Evaluation of LLM-Based Agents.” <https://arxiv.org/abs/2503.16416v1>.
- Yu, Ning, Jie Zhang, Sandeep Mitra, Rebecca Smith, and Adam Rich. 2025. “AI-Educational Development Loop (AI-EDL): A Conceptual Framework to Bridge AI Capabilities with Classical Educational Theories.” <https://arxiv.org/abs/2508.00970v2>.
- Zhang, Gaoyang, Shanghong Zou, Yafang Wang, He Zhang, Ruohua Xu, and Feng Zhao. 2026. “ReusStd-Flow: A Standardized Reusability Framework for Dynamic Workflow Construction in Agentic AI.” <https://arxiv.org/abs/2602.14922v1>.
- Zhang, Guangwei. 2025. “Knowledge Protocol Engineering: A New Paradigm for AI in Domain-Specific Knowledge Work.” <https://arxiv.org/abs/2507.02760v1>.
- Zhang, Linghao. 2026. “Nurture-First Agent Development: Building Domain-Expert AI Agents Through Conversational Knowledge Crystallization.” <https://arxiv.org/abs/2603.10808v1>.
- Zhu, Judy, Dhari Gandhi, Himanshu Joshi, Ahmad Rezaie Mianroodi, Sedef Akinli Kocak, and Dhanesh

- Ramachandran. 2026. “Interpreting Agentic Systems: Beyond Model Explanations to System-Level Accountability.” <https://arxiv.org/abs/2601.17168v1>.
- Zhu, Yizhang, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, et al. 2025. “A Survey of Data Agents: Emerging Paradigm or Overstated Hype?” <https://arxiv.org/abs/2510.23587v2>.
- Zhuge, Mingchen, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, et al. 2024. “Agent-as-a-Judge: Evaluate Agents with Agents.” <https://arxiv.org/abs/2410.10934v2>.